Budgeted Interactive Property Graph Repair with GNN

Anonymous Author(s)

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:

Anonymous Author(s). 2018. Budgeted Interactive Property Graph Repair with GNN. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 NP-Hardness proof

Definition 1.1 (Budget-Constrained GRDG Covering Problem). Given a GRDG graph $\mathcal{G} = (V, F)$ with node difficulties D_v and a set of users $U = \{u_1, \ldots, u_N\}$, each user u_i having a skill value K_i, cognitive capacity CC_i, and cost c_i, and a total budget $B \in \mathbb{R}_{\geq 0}$, find a subset of users $U^* \subseteq U$ and an assignment of disjoint subgraphs $\{P_i \subseteq V \mid u_i \in U^*\}$ such that:

$$U^* = \arg\max_{U' \subseteq U} \sum_{u_i \in U'} \sum_{v \in P_i} EQ(D_v, K_i) \tag{1}$$

subject to:

$$\text{Each } P_i \text{ is connected}, \tag{2}$$

$$\sum_{v \in P_i} D_v \leq CC_i \quad \forall u_i \in U', \tag{3}$$

$$P_i \cap P_j = \emptyset \quad \forall u_i \neq u_j, \tag{4}$$

$$\bigcup_{u_i \in U'} P_i = V, \tag{5}$$

$$\sum_{u_i \in U'} c_i \leq B. \tag{6}$$

where $EQ(D_v, K_i)$ is a component-wise monotonic function defined in Section 4.1 of the paper, estimating the expected quality of a user u_i with skill level K_i on a violation with difficulty D_v.

The above formulation captures the key aspects of the violation assignment task. Constraint (2) ensures that each user is assigned a connected subgraph of violations, preserving local context and facilitating coherent resolution. Constraint (3) guarantees that the total difficulty of assigned violations does not exceed the user's cognitive capacity, reflecting individual workload limits. Constraint (4) enforces disjointness in the assignment, ensuring each violation is handled by at most one user. Constraint (5) ensures that the assignment covers all necessary violations. Finally, constraint (6) is the budget constraint.

Theorem 1.2. *The Budget-Constrained GRDG Covering Problem (bud-GCP) is NP-hard.*

Proof. We reduce the classical Multiple Knapsack problem (MKP) to bud-GCP. An instance of MKP consists of items $I = \{1, \ldots, n\}$, where item i has weight $w_i \geq 0$ and profit $p_i \geq 0$, and knapsacks $M = \{1, \ldots, m\}$, where knapsack j has capacity $C_j \geq 0$. In the decision version of MKP, given a threshold P, the question is whether there exists a packing of each item into at most one knapsack respecting the capacity constraints such that the total profit of the packing is at least P.

Given an instance $\mathcal{I}$ of MKP, we create an instance $\mathcal{J}$ of bud-GCP as follows: create a node v_i for each item $i \in I$ and set $V = \{v_1, \ldots, v_n\}$. Construct a complete graph $G = (V, F)$.

For each node v_i assign the difficulty $D_{v_i} := w_i$. Create a user u_j for each knapsack $j \in M$ and set its cognitive capacity to $CC_j := C_j$. We also create a dummy user u_0 with very large capacity, say $CC_0 := \sum_{i \in I} w_i$. Set $K_j = 1$ for $j \geq 1$ and $K_0 = \varepsilon \in (0, 1)$. Set the user costs $c_i := 1$, $i \geq 0$ and set the budget to $B := m + 1$.

Define the expected-quality function as

$$EQ(D_{v_i}, K_j) = \begin{cases} p_i, & \text{if } K_j = 1, \\ 0, & \text{if } K_j < 1. \end{cases}$$

It is easy to verify that this definition is component-wise monotone and that this reduction is polynomial time in n and m.

We claim that $\mathcal{I}$ is a YES-instance of MKP, i.e., it admits a valid packing of items into knapsacks with total profit at least P subject to budget B iff $\mathcal{J}$ admits a valid assignment of users to violations that covers the entire graph, whose total quality is at least P and the total cost is at most B.

Let $\pi : I \rightharpoonup M$ be a partial function denoting the packing of a subset of items I into the m knapsacks for the MKP instance $\mathcal{I}$. We can construct a corresponding solution to the bud-GCP instance $\mathcal{J}$: assign violation node v_i to user u_j whenever $\pi(i) = j$; for each unassigned item i, assign the violation v_i to the dummy user u_0, i.e., assign a violation v_i to the dummy user u_0 whenever $\pi(i)$ is undefined. Let P_j denote the set of violations assigned to user u_j. Since G is complete, each assigned part P_j is connected, so (2) holds. The cognitive-capacity constraints (3) hold since $\sum_{v \in P_j} D_v = \sum_{i \in P_j} w_i \leq C_j = CC_j$, $j \in \{1, \ldots, m\}$. Disjointness (4) follows from the fact that each item is placed in at most one knapsack and thus each violation is assigned to at most one user. By construction, the assignment covers all violations, satisfying (5). The budget constraint (6) holds by our choice of costs and B. By construction, the bud-GCP objective value equals the MKP profit P.

Conversely, consider a feasible solution for the instance $\mathcal{J}$ of bud-GCP with quality Q and cost $\leq B$. We obtain a feasible MKP

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

item packing as follows: for each user u_j, $j \geq 1$, place item i into knapsack j iff $v_i \in P_j$, i.e., $\pi(i) = j, \forall v_i \in P_j$. Constraints (3) and (4) guarantee that the knapsack capacities are respected and that no item appears in more than one knapsack. Violations assigned to the dummy user, if any, correspond to items left unassigned in MKP. Removing the unassigned nodes doesn't reduce the profit of the item packing since the dummy user contributes 0 quality (and thus 0 profit). Thus, the items assigned to non-dummy users form a feasible MKP packing whose total profit is Q. The theorem follows. □

2 Time Complexity of LARA

Let $|V|$ be the number of nodes of the GRDG, $|F|$ the number of edges, $|U|$ the number of users, $C = |C_i|$ the size of candidates subgraphs per user and T the number of iterations of the algorithm.

Complexity of BuildCandidatesSet. Let Δ be the maximum degree of the GRDG. For user u_i, denote by CC_i the capacity and by D_v the node difficulties. Let $X_i := \min\left\{|V|, \left\lfloor \frac{CC_i}{\min_{v \in V} D_v} \right\rfloor\right\}$ be an upper bound on the number of nodes a candidate subgraphs for u_i can contain. Assume we start from S_i seeds of violations per user and run t_i iterations of 1-exchange local search (add/drop/swap one node) per seed, keeping at most $C_i \leq S_i$ candidate subraphs. The greedy growth step performs at most X_i insertions, checking at most $O(X_i\Delta)$ neighboring nodes to select the one with maximal gain (cost $O(\log n)$), giving $T_{\text{greedy}}(i) = O(X_i\,\Delta\,\log n)$. One step of local search scans a 1-exchange neighborhood of size $O(X_i\Delta)$ with cached deltas; over t_i iterations this is $T(i) = O(t_i\,X_i\,\Delta)$. Hence the per-seed cost is $T_{\text{seed}}(i) = O\big(X_i\,\Delta\,(\log n + t_i)\big)$. With S_i seeds, the complexity for finding the candidate subgraphs for a user is $T_{\text{user}}(i) = S_i \cdot T_{\text{seed}}(i) = O\Big(S_i\,X_i\,\Delta\,(\log n + t_i)\Big)$,

Complexity of SubModularAssignment. For each user i, let C_i be the set of candidate subgraphs produced by BuildCandidatesSet. The total candidates for all the users are $M = \sum_{i=1}^{N} |C_i|$. and the total nodes accross the candidates are $Y = \sum_{i=1}^{N} \sum_{P \in C_i} |P|$. We can leverage an inverted index from each node v for the list of candidate items (i, P) that contain v (of size $O(M)$). Let $c_{\min} = \min_i c_i$ be the minimum cost among the users, and let X_{sel} be the number of selected candidates; clearly $X_{\text{sel}} \leq \min\{M, |U|, \lfloor B/c_{\min} \rfloor\}$.

Computing initial gains $\sum_{v \in P} EQ(D_v, K_i)$ for all (i, P) and building the inverted index costs $O(S)$.

At each iteration of the greedy loop, we extract the current best feasible item in $O(\log M)$ time. Upon choosing $(i^\star, P^\star)$ we (i) remove all other patches of user $i^\star$ (at most $|C_{i^\star}|$ items overall across the run), and (ii) invalidate all candidates that overlap $P^\star$. Using the inverted index, step (ii) takes $O\big(\sum_{v \in P^\star} \Gamma_v\big)$ time, where Γ_v is the number of candidate items containing node v. Because subgraphs are enforced disjoint, each node is invalidated at most once; hence across the whole run $\sum_{\text{selected } P^\star} \sum_{v \in P^\star} \Gamma_v = \sum_v \Gamma_v = Y$. The total number of operations on the lists of candidates is $O(M + X_{\text{sel}})$, so heap work is $O\big((M + X_{\text{sel}}) \log M\big)$.

Summing up, $T_{\text{assignment}} = O\big(Y + (M + X_{\text{sel}}) \log M\big)$. Under uniform parameters ($|C_i| = L$ candidates per user, each of size at most X), we have $M = |U|L$ and $X \leq |U|LX$, giving

$$T_{\text{assignment}} = O\big(|U|LX + (|U|L + X_{\text{sel}}) \log(|U|L)\big).$$

Complexity of the Final Outer Loop. Let T be the maximum number of Lagrangian iterations. The remaining outer-loop work per iteration is lightweight: (i) computing the current primal value Z is $O\big(\sum_{i \in U'} |P_i|\big) \leq O(|V|)$, (ii) the dual quantity UB_t is $O(1)$ once Z and $\sum_{i \in U'} c_i$ are known, (iii) the subgradient update and projection of λ are $O(1)$, and (iv) updating the incumbent $(U^\star, \{P_i^\star\})$ is $O(1)$.

Hence the worst-case total time (without early stopping) is $T_{\text{total}} = T \cdot \Big(|U| \cdot T_{\text{user}} + T_{\text{assignmnet}} + O(|V|)\Big)$. Therefore, T_{total} is

$$O\Big(T \cdot \big(|U|\,S\,X\,\Delta\,(\log|V| + t) + |U|LX + (|U|L + X_{\text{sel}})\log(|U|L) + |V|\big)\Big).$$

In practice the loop halts at some $T' \leq T$ when either the dual–primal gap $UB - Z^\star$ drops below tolerance or the price λ stabilizes; then T above can be replaced by T'.

2.1 Hetero-GAT training

We trained a Heterogeneous Graph Attention Network in PyTorch, using four attention heads and aggregated their outputs via sum concatenation. This way, we obtained the node embeddings. Then, for each edge type, we train a dedicated Multilayer Perceptron classifier that predicts the existence of edges, obtaining the edge confidences. The model was trained for link prediction using the Adam optimizer with a learning rate of 1×10^{-3}, over 50 epochs. For supervision, we used the clean portion of the graph (i.e., subgraph without violations) as the training set, and a set of violations as the test set. The final prediction score for a candidate edge is computed via a sigmoid activation over the output of the per-edge-type classifier, while the embeddings of the nodes were used to compute the node confidence.

3 Balancing the human involvement

In this experiment, we analyzed the influence of GNN involvement and the available budget on the final repair quality.

We vary θ from 0 (the GNN performs all the repairs) to 1 (only humans repair the graph). The budget depends on the dataset, and it varies from 0 (no human can be involved), to the amount necessary to involve all the humans in the repair of all the violations.

Figure 1 illustrates the results. Each graph shows the F1 score as a function of the budget, with different lines corresponding to the employed θ values (e.g., `LARA-0.1` corresponds to $\theta = 0.1$). The reported F1 score is computed as the average between the scores obtained by the GNN and the users. The dashed lines represent the values of θ such that the GNN repairs all the violations.

We can observe a common behavior across the different dataset where the charts are divided in four regions. First, there is the *unfeasible* area, where there are missing points. It corresponds to settings where the budget and the GNN involvement are not enough to assign all the violations. This occurs rarely and only at higher level of θ as more humans need to be involved (e.g., the first two or three steps for `LARA-0.8`). Second, there is the *feasible* region, where given the available budget, all the violations are assigned but the more expert (and costly) users cannot be involved. Then, there is the *inflection point*, that correspond to the amount of budget needed to make the assignment resulting the highest quality (i.e. we can afford to involve high skilled users). Finally, there is the

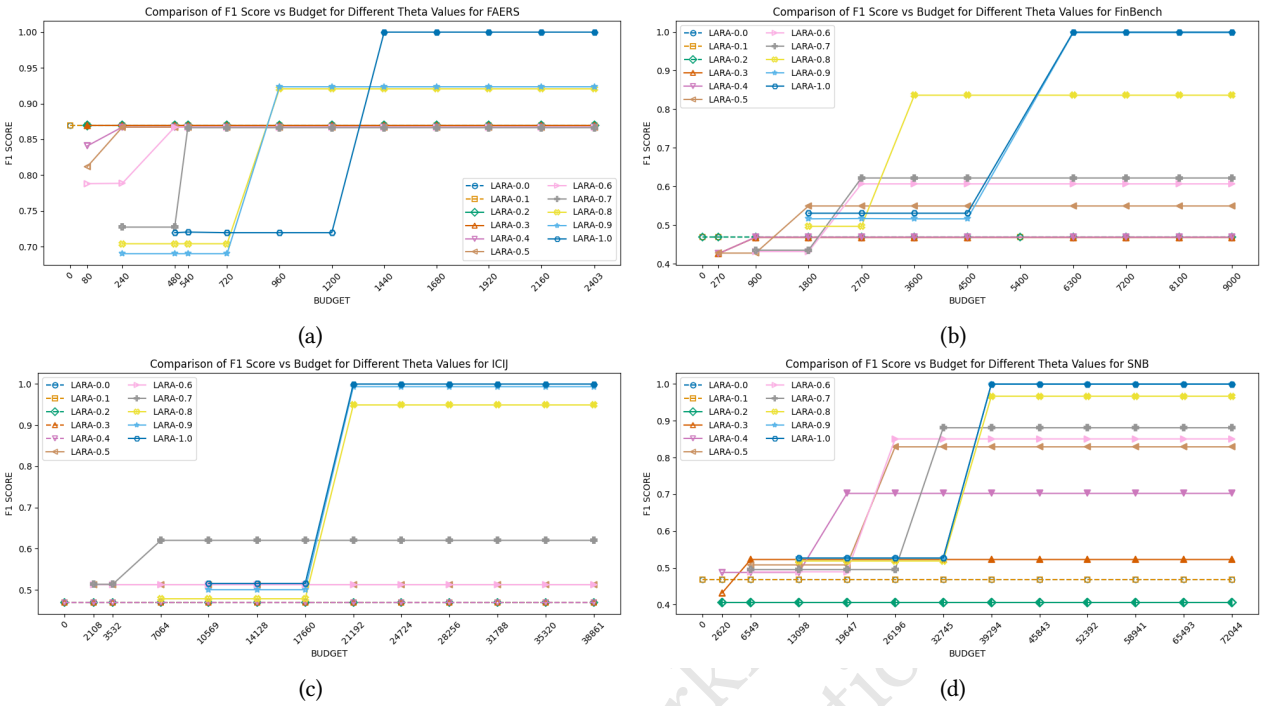

Figure 1: F1-Score of our approach using different θ across different datasets at different budget values.

plateau area, corresponding to cases where augmenting the budget does not increase the quality of the repairs.

For instance, let us consider the case of `LARA-0.8` applied to the `FAERS` dataset (Figure 1(a)). When the budget is set to 0 or 80, the assignment algorithm is unable to allocate a sufficient number of human participants to repair all the violations; this situation corresponds to the *unfeasible* region. When the budget increases to values between 240 and 720, the graph can be repaired; however, not all highly skilled users are involved in the process. As a consequence, the F1 score, remains around 0.7, which characterizes the *feasible* region. The *inflection point* is reached at a budget of 960, where the involvement of additional skilled users results in a improvement of the repair quality, leading to an F1 score of 0.92. Beyond this point, further increases in the budget do not produce significant gains.

It's interesting to observe that for all the datasets, the plateau is reached remaining under the 70% of the total maximum budget. In fact, for F1 scores > 0.8 the the plateau is reached within three or four steps before the max budget. On `ICIJ`, `LARA` exhibits sharper inflection points: with $\theta < 0.6$ fewer than two users are involved, but higher θ drastically increases user demand (2 at `0.6` vs. 263 at `0.7`), leading to a sharp quality rise for $\theta \geq 0.8$.

Notice that some values of θ result in a flat line. This typically occurs for lower values (e.g., `LARA-0.2` on `FinBench` in Figure 1(b)), where the violations predominantly repaired with GNN-Repairs. Because the GNN already fixes the majority, assigning the remaining violations to experts has little impact on overall quality.

For a fixed budget, different θ values yield solutions with varying quality and certified approximation guarantees. To combine them, run LARA concurrently across multiple θ values, apply each instance's stopping rule, and select the one that returns the certified solution. This parallel strategy does not increase the algorithm's asymptotic complexity.

Furthermore, Figure 2 reports the number of repairs performed by users and the GNN, showing how the tunable parameter θ impacts the trade-off between quality and costs highlighted in Table 3 of the paper. Notice that, to achieve a good quality score and outperform the baselines, it is enough to choose the smallest θ that results in more violations assigned to users than to the GNN. In general, for all the datasets, $\theta \in \{0.8, 0.9\}$ represents the best trade-off between automated repairs and user intervention, suggesting that selecting a θ close to the average difficulty value results in a good quality/cost balance.

Finally, Figure 3 reports the runtime of `LARA` for each θ under the maximum budget. Being sparse, our graphs lead to near log-linear complexity in the number of nodes. Runtime increases with θ due to the larger number of assigned violations, but remains moderate. In contrast, the F1-score, discussed next, increases monotonically with θ.

3.1 LLMs for Data Repairs

To further investigate the repair process, we evaluate whether Large Language Models (LLMs) can partially or fully replace human workers for repairing violations, analyzing the trade-off between repair quality and monetary cost when delegating repairs to LLMs instead of users. We consider three values of $\theta \in 0.5, 0.8, 1.0$, corresponding to `LARA-`θ. Increasing θ reduces the number of violations automatically repaired by the GNN, leaving a larger portion of difficult

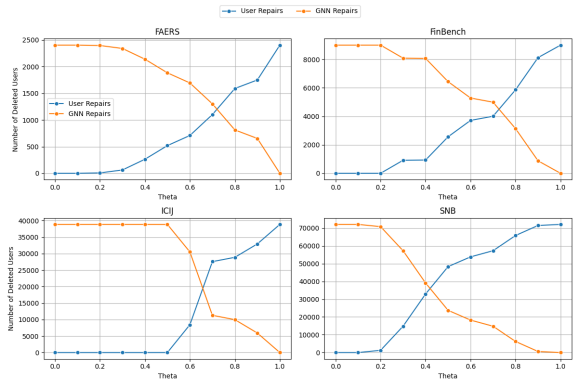

Figure 2: Number of repairs done by the GNN and by the users at different θ.

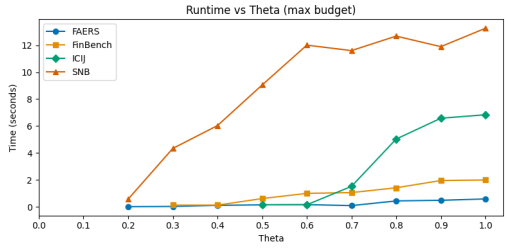

Figure 3: Runtime of LARA at different θ. Missing points correspond to run where only the GNN repaired the graphs.

violations to be handled externally. For each θ, we evaluate two repair strategies: (i) assigning all remaining violations to the LLM, and (ii) splitting the remaining violations between the LLM and human users according to the violation difficulty score. More specifically, for $\theta = 0.5$, we first assign all violations with difficulty in $[0.5, 1]$ to the LLM, and then evaluate a hybrid strategy where violations in $[0.5, 0.75]$ are repaired by the LLM while those in $[0.75, 1]$ are repaired by users. Similarly, for $\theta = 0.8$, we evaluate both a full-LLM strategy on violations in $[0.2, 1]$ and a hybrid strategy where violations in $[0.2, 0.6]$ are assigned to the LLM and the remaining ones to users. Finally, for $\theta = 1.0$, we compare a fully LLM-based strategy on all violations with a hybrid configuration where violations in $[0, 0.5]$ are repaired by the LLM and the remaining ones by users. The LLM used in our experiments is Llama-3-8B[1]. Table 1 reports the resulting F1-score and total repair cost for each dataset and configuration. We additionally report the relative increase (↑) or decrease (↓) in F1-score with respect to the original LARA approach without LLM-based repairs. For the LLM's cost we ketp 0 (as for the GNN), while for users we maintained the same settings described in Section 5.1 of the paper. Overall, the results show that fully replacing users with LLMs substantially reduces the repair cost, often eliminating it entirely, but at the expense of repair quality. In almost all configurations, the F1-score decreases compared to the original LARA approach without LLM-based repairs, with reductions ranging from moderate to severe depending on the dataset and the value of θ. Only a few configurations with $\theta = 0.5$ achieve improvements over the original approach, namely on FinBench and ICIJ, suggesting that LLMs can sometimes effectively repair medium-difficulty violations when the GNN already filters out the easier cases. Hybrid strategies combining LLMs and users consistently perform better than fully LLM-based repairs, but still generally remain below the original fully human-assisted configuration. We additionally conducted experiments using GPT-5 on FAERS and FinBench only, due to budget constraints. Compared to Llama 3 8B, GPT-5 consistently improved the repair quality across all tested θ values, yielding gains of approximately 0.05-0.10 in F1-score. For instance, with $\theta = 1.0$, the fully LLM-based configuration achieved F1-scores of 0.406 and 0.605 on FAERS and FinBench, respectively, while the hybrid configuration with 50% user involvement reached 0.608 and 0.895. Nevertheless, even with GPT-5, the repair quality generally remained below the original LARA approach, confirming that current LLMs are still less reliable than human users for complex data repair tasks.

[1]https://huggingface.co/meta-llama/Meta-Llama-3-8B

3.2 Testing Different Constraint Types

To further evaluate the generality of our approach, we conducted an additional study using different classes of integrity constraints. In particular, we considered three representative categories: a general denial constraint (DC), a non-key functional dependency (FD), and a key constraint (Key). These constraints were instantiated on the two real-world datasets FAERS and ICIJ. The complete list of constraints is reported in Section 5. We reran the full end-to-end repair pipeline while varying both the confidence threshold θ and the repair budget. Figure 4 shows the F1-score for different budgets and values of θ for LARA. Overall, the results confirm that the trends observed in Section 3, with a plateau reached for higher values of θ (> 0.7) and without using the full available budget. Figures 5 instead, breaks down the results for each type of constraint (DC, FD and Key) reporting the F1-score for the main operating regions, namely $\theta > 0.5$ and budget values corresponding to the plateau regions observed in Figure 4. For FAERS, denial constraints and key constraints benefit the most from larger budgets, reaching near-perfect repair quality when sufficient resources are available. Functional dependencies exhibit more stable behavior across different parameter settings, with moderate F1-scores and relatively limited sensitivity to θ. We also observe that higher-quality repairs are typically associated with increased repair costs, especially for key constraints at large budgets, reflecting the higher complexity of resolving structurally critical inconsistencies. For ICIJ, the behavior is even more stable across parameter configurations. Both denial constraints and key constraints achieve very high F1 scores for medium and large budgets, while functional dependencies maintain slightly lower but highly consistent performance. In contrast to FAERS, the repair costs in ICIJ remain relatively stable across values of θ, indicating that the assignment strategy adapts well to different constraint semantics without requiring significant additional repair effort. Overall, the key constraints seem harder to solve, since a good F1-score is reached only with the full amount of users ($\theta = 1$) and indeed are the most expensive. FDs instead seem to require less budget and reach a lower accuracy; this suggests that

Theta	LLM	F1 Score				Total Cost			
		FAERS	FinBench	ICIJ	SNB	FAERS	FinBench	ICIJ	SNB
0.5	0.5-1	0.335 ↓ 65%	0.245 ↓ 55%	0.629 ↑ 23%	0.445 ↓ 47%	0	0	0	0
	0.5-0.75	0.545 ↓ 38%	0.785 ↑ 42%	0.900 ↑ 75%	0.807 ↓ 3%	57.33	866.7	0.385	18229
0.8	0.2-1	0.335 ↓ 64%	0.277 ↓ 67%	0.613 ↓ 36%	0.439 ↓ 55%	0	0	0	0
	0.2-0.6	0.545 ↓ 41%	0.787 ↓ 5%	0.867 ↓ 9%	0.835 ↓ 14%	191.73	2313	12844	23714
1	0-1	0.335 ↓ 66%	0.262 ↓ 74%	0.605 ↓ 39%	0.445 ↓ 55%	0	0	0	0
	0-0.5	0.545 ↓ 45%	0.762 ↓ 24%	0.897 ↓ 10%	0.857 ↓ 15%	310.2	3514	14464	26580

Table 1: Comparison of F1 score and total cost for each dataset with different values of *theta* and LLM involvement.

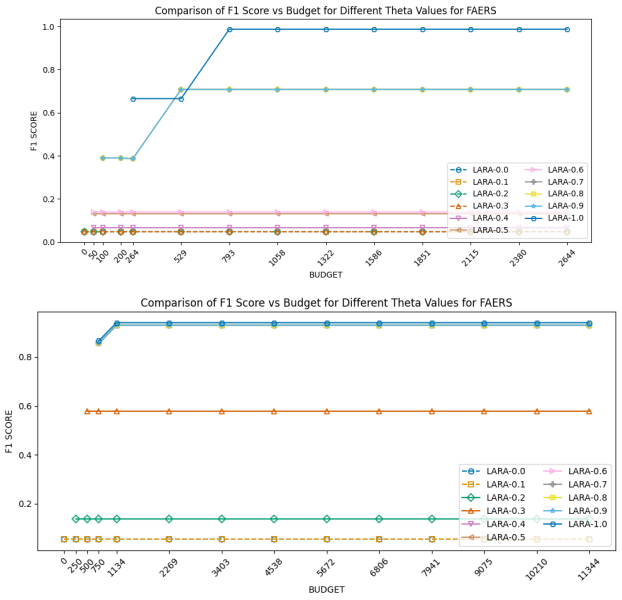

Figure 4: F1-Score of our approach using different θ for FAERS

GNNs tend to underestimate their difficulty resulting in less qualitative results. Generally, these results suggest that the proposed framework is not restricted to a single type of integrity constraint. Although the experiments are implemented using GDCs as the unifying formalism, the behavior of the assignment and repair pipeline remains robust across keys, functional dependencies, and general denial constraints.

4 Constraints

In this Section we list the contraints used to introduce the violations for each dataset.

4.1 LDBC Finbench

An account cannot transfer money if it is blocked

$$\phi_1 := (Transfer(X, Y), X.isBLocked = True \rightarrow \bot) \quad (7)$$

A guarantor needs to have more than 5M in the bank account

$$\phi_2 := (Guarantee(X, Y) \wedge Own(Y, Z), \quad (8)$$
$$Z.balance < 500000000 \wedge id(X) \neq id(Y) \rightarrow \bot) \quad (9)$$

A person cannot be guarantor of themself

$$\phi_3 := (Guarantee(X, Y), id(X) = id(Y) \rightarrow \bot) \quad (10)$$

Interest Rates for company need to be greater than 0.2

$$\phi_4 := (Apply(X, Y), Y.interestRate < 0.2 \rightarrow \bot) \quad (11)$$

4.2 LDBC SNB

A person cannot know themself

$$\phi_1 := (Knows(X, Y), id(X) = id(Y) \rightarrow \bot) \quad (12)$$

Two people that know each other need to live in the same city

$$\phi_2 := (Knows(X, Y), id(X) <> id(Y) \quad (13)$$
$$\wedge X.LocationCityId \neq Y.LocationCityId \rightarrow \bot) \quad (14)$$

A user cannot like their own post

$$\phi_3 := (CREATED(po, p) \wedge LIKES(po, p) \rightarrow \bot) \quad (15)$$

A person has to live in the same country as the organization they work for.

$$\phi_4 := (LIVES_IN(pl1, p) \wedge WORK_AT(p, o) \wedge LOCATED_IN(o, pl) \quad (16)$$
$$id(pl) \neq id(pl1) \rightarrow \bot) \quad (17)$$

4.3 FAERS (Real)

A drug cannot be primary suspect and secondary suspect of the same case

$$\phi_1 := (IS\_PRIMARY\_SUSPECT(X, Y) \quad (18)$$
$$\wedge IS\_SECONDARY\_SUSPECT(Y, Z), \quad (19)$$
$$id(X) = id(Y) \rightarrow \bot) \quad (20)$$

A drug cannot be primary suspect of itself

$$\phi_1 := (IS\_PRIMARY\_SUSPECT(X, Y) \quad (21)$$
$$id(X) = id(Y) \rightarrow \bot) \quad (22)$$

A case cannot fall under two different age groups

$$\phi_3 := (FALLS\_UNDER(X, Y) \wedge FALLS\_UNDER(X, Z), \quad (23)$$
$$id(Z) \neq id(Y) \rightarrow \bot) \quad (24)$$

A drug cannot be prescribed to a child

$$\phi_3 := (PRESCRIBED(X, Y) \wedge RECEIVED(Y, Z) \quad (25)$$
$$\wedge FALLS\_UNDER(Z, W) \quad (26)$$
$$W.ageGroup = "Child" \rightarrow \bot) \quad (27)$$

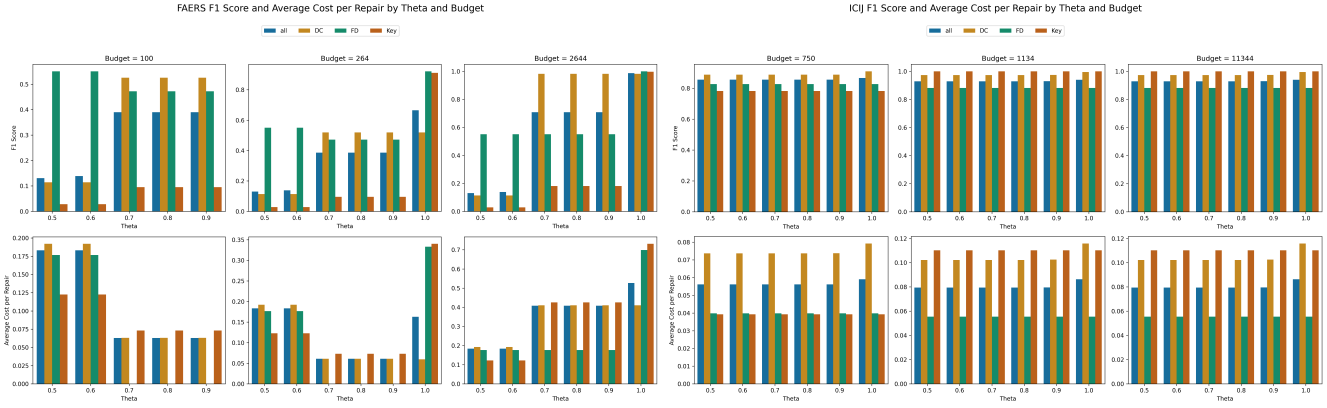

Figure 5: F1-Scores and total costs of our approach using $\theta > 0.5$ across FAERS and ICIJ at different budget values.

4.4 ICIJ (Real)

The registered address of an entity must be the same as the one of the address

$$\phi_1 := (registered\_address(X, Y), \tag{28}$$
$$X.country\_codes \neq Y.country\_codes \rightarrow \bot) \tag{29}$$

An address cannot be registered both by an office and an entity

$$\phi_2 := (OFFICER\_OF(o, e) \wedge REGISTERED\_ADDRESS(e, a) \tag{30}$$
$$\wedge\ REGISTERED\_ADDRESS(o, a) \rightarrow \bot) \tag{31}$$

A shareholder needs to have a minimum amount of money

$$\phi_3 := SHAREHOLDER\_OF(o, e) \tag{32}$$
$$WHERE\ o.networth < 17000000, \rightarrow \bot) \tag{33}$$

An officer cannot be both intermediary and shareholder for the same entity

$$\phi_4 := (INTERMEDIARY\_OF(i, e) \tag{34}$$
$$\wedge\ SHAREHOLDER\_OF(i, e) \rightarrow \bot) \tag{35}$$

5 Different Type of Constraints

5.1 ICIJ

Denial Constraint: An officer relation cannot end after the entity closed date

$$\phi_5 := (OFFICER\_OF(o, e) \wedge HAS\_CLOSED\_DATE(e, d), \tag{36}$$
$$o.end\_date > d.value \rightarrow \bot) \tag{37}$$

Key Constraint: Two different entities in the same jurisdiction cannot have the same name and incorporation date

$$\phi_6 := (HAS\_JURISDICTION(e, j) \wedge HAS\_JURISDICTION(e_1, j), \tag{38}$$
$$id(e) \neq id(e_1) \wedge e.incorporation\_date = e_1.incorporation\_date \tag{39}$$
$$\wedge\ e.name = e_1.name \rightarrow \bot) \tag{40}$$

Functional Dependency: The country associated with an entity must be the same as the country of its registered address

$$\phi_7 := (rr(c_1, n) \wedge REGISTERED\_ADDRESS(n, a) \tag{41}$$
$$\wedge\ LOCATED\_IN(a, c), \tag{42}$$
$$c_1.code \neq c.code \rightarrow \bot) \tag{43}$$

5.2 FAERS (Real)

Key Constraint: Two different cases cannot have the same event date, age, and gender

$$\phi_5 := (HAS\_EVENT\_DATE(c, d) \wedge HAS\_AGE(c, a) \tag{44}$$
$$\wedge\ HAS\_GENDER(c, g) \wedge HAS\_EVENT\_DATE(c_1, d) \tag{45}$$
$$\wedge\ HAS\_AGE(c_1, a) \wedge HAS\_GENDER(c_1, g), \tag{46}$$
$$id(c) \neq id(c_1) \rightarrow \bot) \tag{47}$$

Functional Dependency: The age of a case determines the corresponding Age Group

$$\phi_6 := (HAS\_AGE(c, a) \wedge FALLS\_UNDER(c, ag), \tag{48}$$
$$a.value < ag.minAge \vee a.value > ag.maxAge \rightarrow \bot) \tag{49}$$

Denial Constraint: A drug exposure cannot appear as both primary suspect and secondary suspect with the same drug name and dose amount in the same case

$$\phi_7 := (HAS\_DRUG\_EXPOSURE(c, d) \wedge HAS\_ROLE(d, r) \tag{50}$$
$$\wedge\ HAS\_DOSE\_AMOUNT(d, a) \tag{51}$$
$$\wedge\ HAS\_DRUG\_EXPOSURE(c, d_1) \wedge HAS\_ROLE(d_1, r_1) \tag{52}$$
$$\wedge\ HAS\_DOSE\_AMOUNT(d_1, a_1), \tag{53}$$
$$r.value = \text{“}Primary\ Suspect\text{”} \tag{54}$$
$$\wedge\ r_1.value = \text{“}Secondary\ Suspect\text{”} \tag{55}$$
$$\wedge\ d.drugName = d_1.drugName \tag{56}$$
$$\wedge\ id(d) \neq id(d_1) \tag{57}$$
$$\wedge\ a.value = a_1.value \rightarrow \bot) \tag{58}$$

References

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009